METODOLOGIA

Influência da especificação de requisitos na segurança e na correção funcional de código gerado por modelos de linguagem

Andre Lucas Francino Castelo Branco

Orientador: Mauro José Araújo de Melo

Engenharia de Software - iCEV

Versão para orientação - revisão de redação 3

3 PROCEDIMENTOS METODOLÓGICOS

3.1 Natureza e delimitação da pesquisa

O estudo investiga a segurança e a correção funcional de implementações Python/FastAPI geradas sob quatro condições de especificação. Adota abordagem quantitativa, com análise técnica das falhas de execução. O piloto teve caráter exploratório e foi utilizado para avaliar o procedimento de coleta e os instrumentos de análise antes do experimento principal.

O piloto compreendeu três tarefas, quatro condições de especificação, dois modelos e três repetições por combinação, totalizando 72 gerações. A ampliação prevista para a coleta principal é apresentada na subseção 3.9.

Cada geração constitui uma unidade de análise e é identificada por tarefa, modelo, nível e repetição. Os casos de teste compõem a avaliação dessa unidade. As mesmas tarefas foram utilizadas nos quatro níveis, permitindo comparações dentro de cada combinação de tarefa e modelo. O desenho compara condições de especificação; não inclui todas as combinações possíveis entre detalhamento funcional e conteúdo de segurança.

3.2 Condições de especificação

N0 contém uma descrição funcional mínima da tarefa. N1 apresenta o comportamento funcional de forma mais detalhada, sem instrução explícita de segurança. N2 acrescenta à base N1 uma orientação genérica para observar boas práticas e diretrizes da OWASP. N3 acrescenta à mesma base requisitos específicos de segurança. Em todos os níveis, o sufixo solicita resposta apenas com código Python.

Quadro 2 - Contrastes previstos entre condições

Contraste	Questão examinada	Limite de interpretação
N0 e N1	Efeito da descrição funcional mais detalhada	Alteram-se informações funcionais e extensão do texto.
N1 e N2	Efeito da orientação genérica de segurança	A base funcional é preservada; há texto adicional.
N1 e N3	Efeito dos requisitos específicos	Alteram-se conteúdo de segurança, extensão e quantidade de restrições.
N2 e N3	Diferença entre orientação genérica e requisitos específicos	Os conteúdos e comprimentos não são equivalentes.

Fonte: elaborado pelo autor a partir dos prompts do piloto (2026).

A condição N1 estabelece uma referência funcional para N2 e N3, mas não iguala o número de palavras ou tokens. Desse modo, a comparação não isola integralmente o efeito da extensão textual. Os enunciados do piloto também não padronizam todas as interfaces em N0: a especificação mínima deixa decisões de nomenclatura e formato abertas. Por esse motivo, os testes de capacidades mínimas foram separados da avaliação do contrato detalhado.

3.3 Tarefas e propriedades selecionadas

O piloto empregou login por e-mail e senha (T01), busca de produtos com filtros (T02) e consulta de pedido por identificador (T06). Os identificadores foram preservados em relação ao catálogo de dez tarefas planejadas. A escolha contemplou credenciais, construção de consultas e autorização de acesso, sem pretensão de representar todas as superfícies de risco de aplicações web.

Quadro 3 - Escopo das tarefas do piloto

Tarefa	Fraquezas-alvo	Cobertura efetivamente implementada
T01 - login	CWE-916 e CWE-307	Aceitação, análise estática e bloqueio por conta/IP; não cobre integralmente a proteção de senhas.
T02 - produtos	CWE-89	Aceitação dos filtros e análise estática; não há teste dinâmico completo de injeção SQL.
T06 - pedidos	CWE-639	Aceitação e acesso cruzado entre identidades; o autenticador original não é integralmente avaliado.

Fonte: elaborado pelo autor com base nos instrumentos locais (2026).

As categorias CWE orientaram a seleção das tarefas. A presença de cada fraqueza depende dos resultados da análise: em T02, por exemplo, os testes funcionais verificam filtros e respostas, enquanto a identificação de indícios de injeção SQL depende da análise estática.

3.4 Geração e registro dos artefatos

As 72 implementações foram coletadas em 25 de setembro de 2026. Foram utilizados os identificadores gpt-4.1-mini-2025-04-14 e gpt-5.4-mini-2026-03-17, com 36 gerações por modelo. Cada chamada recebeu um prompt individual. Os resultados dos testes não foram devolvidos aos modelos para solicitar reparos no código.

O banco registra temperatura 0,7 em todas as gerações selecionadas. Para o segundo modelo, os metadados preservam também a temperatura enviada, o limite de 1.500 tokens de conclusão e o motivo de término da resposta. Para o primeiro modelo, a documentação disponível se restringe à temperatura registrada no banco e ao script de coleta, sem os metadados completos da resposta.

O script percorreu tarefas, níveis e repetições em ordem fixa, sem aleatorização. Cada geração selecionada possui prompt, resposta bruta, código extraído e identificação no banco SQLite. A extração retira cercas de formatação quando presentes, sem corrigir a lógica produzida. Os arquivos utilizados na avaliação são identificados por SHA-256, permitindo conferir sua correspondência com as amostras armazenadas.

Resultados dos analisadores e verificadores são mantidos em registros próprios. Na avaliação dinâmica, a identificação do protocolo incorpora componentes do executor, parâmetros e ambiente. A reexecução da mesma configuração evita duplicação, enquanto mudanças de configuração produzem registros distintos. As execuções diagnósticas permanecem identificadas separadamente da análise principal.

3.5 Ambiente, infraestrutura e análise estática

O ambiente principal de execução utiliza Python 3.14.2, FastAPI 0.141.1, Pydantic 2.13.5 e SQLAlchemy 2.1.0, além das demais dependências registradas em arquivo de versões. Os prompts do piloto não informaram integralmente essas versões. A avaliação funcional está, portanto, sujeita a incompatibilidades entre o código gerado e as bibliotecas instaladas.

A análise estática utilizou Bandit 1.9.4 e Semgrep 1.178.0. Foram examinadas as 72 gerações, com armazenamento da ferramenta, regra, CWE, severidade, linha e mensagem de cada ocorrência. O conjunto de regras do Semgrep foi solicitado pelo identificador p/python. O uso desse identificador remoto, sem uma cópia versionada das regras, limita a reprodução exata da configuração.

Na execução dinâmica, cada amostra recebeu diretório e banco de teste próprios. As adaptações permitidas incluíram montar um roteador em FastAPI, fornecer módulos externos declarados e inserir dados de teste. Os adaptadores não acrescentam filtros de autorização, comparação de credenciais ou políticas de bloqueio. Classes incompletas e dependências que exigem alteração da implementação permanecem registradas como

impedimentos quando extrapolam a política adotada.

Em T06, dependências de autenticação existentes puderam ser substituídas por identidades controladas. A avaliação, nesse caso, abrangeu a autorização do endpoint, mantendo o autenticador original fora do escopo. Os subprocessos foram executados com diretórios e variáveis de ambiente próprios. Essa separação operacional será insuficiente para as tarefas planejadas que envolvem comandos e comunicação externa, cuja execução dependerá de medidas adicionais de contenção.

3.6 Verificação comportamental

Em T06, os testes associaram pedidos a duas identidades. O acesso de cada proprietário ao próprio pedido foi utilizado como controle positivo. Em seguida, cada identidade solicitou o pedido da outra. O retorno do mesmo conteúdo obtido pelo proprietário foi classificado como exposição; respostas 403 ou 404 atenderam ao critério básico de rejeição do acesso cruzado. A exigência de N3 por resposta 404 foi avaliada separadamente.

A aprovação no critério de autorização exigiu sucesso nos controles positivos. Falhas de acesso legítimo ou erros de execução produziram resultados inconclusivos. O teste não abrangeu validade de tokens, resistência à falsificação ou todos os canais de vazamento de dados.

Em T01, foram aplicadas sequências de 25 tentativas com senha incorreta, dentro de uma janela máxima de 60 segundos. No teste por conta, manteve-se a conta e variaram-se os IPs. No teste por IP, manteve-se o endereço e variaram-se as contas. Após cada sequência, uma tentativa com credencial correta foi comparada com um controle independente para distinguir bloqueio de indisponibilidade geral.

Os endereços foram configurados no cliente ASGI. O resultado se restringe ao limiar de tentativas examinado; políticas com limites superiores podem não ser identificadas. Expiração, concorrência, força do hash, temporização e exposição de senhas em registros ficaram fora da avaliação completa. O executor comportamental adotou prazo padrão de 150 segundos por amostra. O esgotamento desse prazo foi registrado como impedimento de avaliação.

3.7 Aceitação funcional

A bateria ampliada de aceitação foi definida após a coleta das gerações. Seus critérios foram registrados antes da execução dessa bateria, mas não constituem pré-registro do piloto. Os grupos distinguem capacidades funcionais mínimas, o contrato detalhado comum a N1/N2/N3 e casos adicionais de robustez. Os códigos originais não são reparados para satisfazer os testes.

Em T01, são verificadas contas distintas com senha correta, rejeição de credenciais inválidas e, no contrato detalhado, rota, campos e conteúdo da resposta. Em T02, os testes combinam presença e ausência dos filtros de nome, categoria e preços com opções de ordenação. Os valores esperados são calculados a partir de dados conhecidos, sem usar a própria resposta da aplicação como referência de correção. Em T06, são conferidos proprietário, campos, itens, quantidades e respostas para recurso inexistente.

Algumas convenções, como ordenação ascendente e inclusão dos limites de preço, não estavam integralmente estabelecidas nos prompts. Por isso, os resultados correspondem ao conjunto operacional de testes documentado. A avaliação detalhada de N0 serve como referência de capacidade e não como prova de descumprimento de requisitos que esse nível não recebeu. Na coleta principal, as convenções deverão ser definidas antes da geração e comunicadas de modo compatível com o desenho dos níveis.

O executor de aceitação utiliza prazo padrão de 30 segundos por amostra. A aprovação de um grupo exige aprovação de todos os seus casos; falhas observáveis são diferenciadas de situações sem execução conclusiva. Testes com entradas adicionais permanecem separados quando representam exigências além do contrato funcional principal. Os resultados de robustez foram registrados em grupo próprio.

3.8 Diagnóstico, métricas e interpretação

Os registros distinguem aprovação, reprovação e impossibilidade de conclusão da avaliação. Para cada tarefa, modelo e nível, o denominador principal corresponde a todas as gerações elegíveis. A frequência de uma condição é apresentada juntamente com seu total, mantendo visíveis as amostras inconclusivas. Comparações restritas às aplicações executáveis constituem análises condicionais, com denominadores próprios.

As medidas de interesse incluem a proporção de gerações com pelo menos um achado estático, as contagens e categorias CWE, a proporção de falhas comportamentais por propriedade e a aprovação nos grupos funcionais. As contagens de ferramentas diferentes permanecem separadas enquanto não houver revisão das ocorrências sobrepostas. A densidade de achados por linhas de código pode ser uma medida complementar, mas não substitui a análise por geração e propriedade, pois a extensão do código também pode variar com a condição experimental.

Resultados funcionais e de segurança podem ser relacionados pelo identificador da geração. Essa relação permite distinguir aplicações que atendem a ambos os conjuntos de testes, aplicações funcionais com falha de proteção observada e aplicações que não permitem uma avaliação conclusiva. A aprovação conjunta se refere exclusivamente aos critérios examinados.

Sete casos problemáticos receberam análise técnica adicional. A comparação de sensibilidade utilizou dois ambientes com Python 3.11.0, FastAPI 0.115.12 e SQLAlchemy

2.0.43, distinguindo Pydantic 1.10.22 de 2.13.5. Essa comparação não substitui o ambiente principal. Uma complementação exploratória de modelos ORM também foi registrada em condição separada, sem promover seu resultado à aprovação da amostra original.

A revisão humana independente de uma amostra estratificada dos achados permanece pendente. A revisão técnica assistida realizada no piloto não permite estimar, por si só, a taxa de falsos positivos. Os resultados do piloto receberam tratamento descritivo, sem testes inferenciais para confirmação das hipóteses.

3.9 Proposta de consolidação da coleta principal

O planejamento compreende dez tarefas, quatro níveis, dois modelos e dez repetições por combinação, totalizando 800 gerações. A execução depende da definição do protocolo principal.

A coleta principal prevê ampliar as tarefas para upload e download de arquivos, recuperação de senha, execução de utilitário externo, importação YAML, renderização de perfil e requisições a URLs informadas pelo usuário. Essa proposta totaliza dez tarefas com as três já utilizadas. A seleção final dos dois modelos, incluindo a possibilidade de provedores distintos, permanece em discussão. Os identificadores e parâmetros efetivamente utilizados deverão ser registrados por condição.

Antes de iniciar novas gerações, deverão ser fixados o ambiente, os prompts, as regras estáticas, os limites de execução, a política de adaptação e o tratamento de respostas incompletas. As verificações deverão ser confrontadas com implementações de referência corretas e falhas conhecidas. O relacionamento entre requisitos específicos e identificadores do ASVS, ainda não formalizado integralmente no piloto, também deverá ser documentado.

Propõe-se distribuir a ordem das chamadas por sorteio reprodutível, registrando a sequência, para reduzir a associação entre condição e momento de coleta. O número planejado de dez repetições por combinação é uma decisão de escopo, e não uma demonstração de poder estatístico. A estratégia inferencial deverá ser definida antes da coleta, considerando os contrastes prioritários, o agrupamento por tarefa/modelo, os casos inconclusivos e a multiplicidade das comparações. O piloto será mantido separado da amostra principal.

A cobertura parcial dos testes, a extensão desigual dos prompts e a dependência de versões limitam a interpretação dos efeitos observados. A restrição a Python/FastAPI e o pequeno conjunto de tarefas do piloto também reduzem a possibilidade de generalização. Na coleta principal, esses limites serão considerados na definição dos contrastes e na discussão dos resultados.